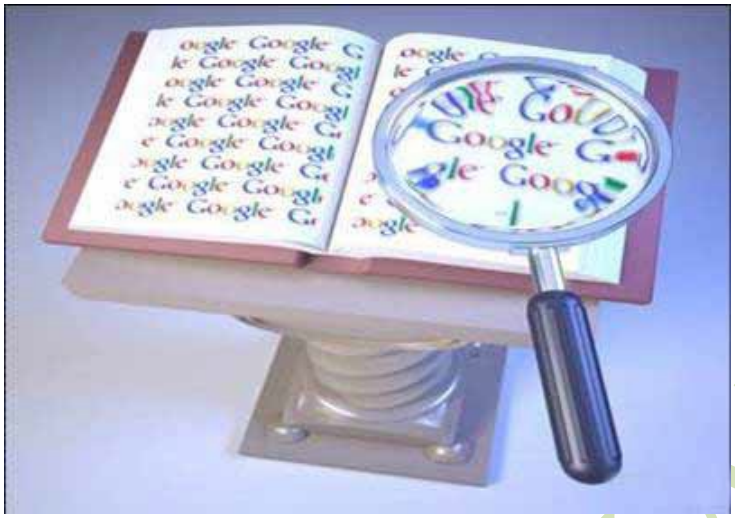
BÀI 6: TÌM KIẾM



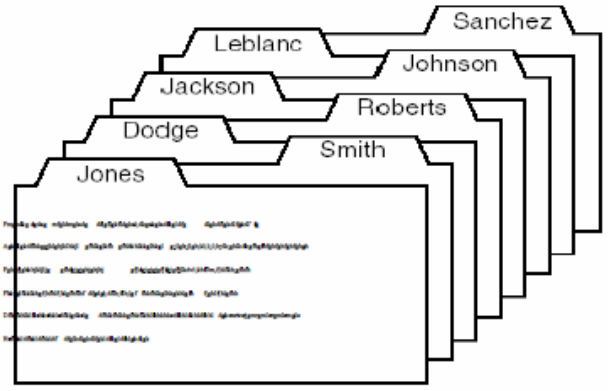
Mục tiêu	Nội dung
Sau khi học bài này, các bạn có thể: • Mô tả đúng nội dung bài toán tìm kiếm và hướng giải quyết bài toán tìm kiếm. • Trình bày và thực hiện cài đặt các thuật toán tìm kiếm như tìm kiếm tuần tự, tìm kiếm nhị phân... một cách chính xác thông qua ngôn ngữ lập trình C. • Mô tả đúng về cây nhị phân tìm kiếm. Trình bày và cài đặt các thao tác trên cây nhị phân tìm kiếm một cách chính xác. • Sử dụng các giải thuật tìm kiếm thích hợp để giải quyết các bài toán trong thực tế.	• Bài toán tìm kiếm. • Tìm kiếm tuần tự. • Tìm kiếm nhị phân. • Cây nhị phân tìm kiếm.

Thời lượng học
10 tiết

6.1. Bài toán tìm kiếm:

Trong hầu hết các hệ thống tin học, khi khai thác, xử lý dữ liệu chúng ta đều phải thực hiện thao tác tìm kiếm và xử lý thông tin. Chẳng hạn trong hệ thống tra cứu điểm thi, việc tra cứu, tìm kiếm kết quả thi của thí sinh diễn ra rất nhanh chóng và chính xác. Thí sinh chỉ cần truy cập vào hệ thống tra cứu điểm thi, rồi nhập thông tin của mình như họ tên, ngày tháng năm sinh hoặc số báo danh là hệ thống nhanh chóng đưa ra kết quả thi của thí sinh đó mà không cần tìm đến trường mình dự thi để tìm hiểu kết quả thi của mình. Đây là một trong rất nhiều ứng dụng của bài toán tìm kiếm trong các hệ thống tin học. Trong bài này, chúng ta sẽ đi tìm hiểu về bài toán tìm kiếm và nghiên cứu đánh giá các thuật giải tìm kiếm để đưa ra được giải thuật phù hợp nhất với yêu cầu của bài toán đặt ra.

Tìm kiếm luôn là thao tác nền móng cho rất nhiều tác vụ tính toán. Tìm kiếm nghĩa là tìm một hay nhiều mẩu thông tin đã được lưu trữ. Thông thường, thông tin được chia thành các mẩu tin (record), mỗi mẩu tin đều có một khóa (key) dùng cho việc tìm kiếm. Ta sẽ luôn có một khoá cho trước giống như khoá của các mẩu tin mà ta cần tìm. Mỗi mẩu tin được tìm thấy sẽ chứa toàn bộ thông tin để cung cấp cho một quá trình xử lý nào đó.



Hình 6.1. ***Mẩu tin và khóa***

Khi đó, bài toán tìm kiếm tổng quát có thể được phát biểu như sau:

"Cho một bảng gồm n mẩu tin R_1, R_2,…, R_n. Với mỗi mẩu tin R_i được tương ứng với một khóa k_i. Hãy tìm mẩu tin có giá trị khóa bằng X cho trước."

Công việc tìm kiếm sẽ hoàn thành nếu có một trong hai tình huống sau sảy ra:

- Tìm được mẩu tin có khóa tương ứng bằng X, lúc đó phép tìm kiếm thành công.
- Không tìm được mẩu tin nào có khóa tìm kiếm bằng X, phép tìm kiếm thất bại.

Ví dụ 6.1.

Một Ngân hàng nắm giữ tất cả thông tin của rất nhiều tài khoản khách hàng và cần tìm kiếm để kiểm tra các biến động. Một hãng Bảo hiểm hay một hệ thống trợ giúp bán vé xe, vé máy bay… Việc tìm kiếm thông tin để đáp ứng việc xắp đặt ghế và các yêu cầu tương tự như vậy là thực sự cần thiết.

Tìm kiếm thường là tác vụ hay được sử dụng trong các chương trình và cũng là tác vụ tốn nhiều thời gian nhất.Vì thế, việc tổ chức cấu trúc dữ liệu và đưa ra giải thuật tìm kiếm phù hợp nhất có ảnh hưởng rất lớn đến hiệu suất của chương trình.

Trong việc mô tả cấu trúc dữ liệu của việc tìm kiếm, thuật ngữ thường được dùng là từ điển và **bảng ký hiệu**. Một ví dụ điển hình như ta xây dựng hệ thống tra từ điển Tiếng Anh chẳng hạn. Ở đây, "khoá" là từ và "mẩu tin" là diễn giải nghĩa cho từ đó. Mỗi mẩu tin chứa định nghĩa, cách phát âm và các thông tin khác. Bảng ký hiệu chính là từ điển cho chương trình và các mẩu tin chứa thông tin mô tả đối tượng được đặt tên.

Giải thuật tìm kiếm là một thuật toán lấy đầu vào là một bài toán và trả về kết quả là một lời giải cho bài toán đó, thường là sau khi cân nhắc giữa một loạt các lời giải có thể. Cách tốt nhất để tìm ra các thuật toán tìm kiếm là đưa ra các thao tác tổng quát sao cho ta có thể áp dụng chúng cho các bài toán tìm kiếm khác nhau. Các thao tác đó gồm:

- Khởi tạo cấu trúc dữ liệu (INITIALIZE).
- Tìm kiếm một hay nhiều mẩu tin có khoá đã cho (SEARCH).
- Chèn thêm một mẩu tin mới (INSERT).
- Nối lại từ điển để tạo thành một từ điển lớn hơn (JOIN).
- Sắp xếp từ điển; xuất ra tất cả các mẩu tin theo thứ tự được sắp xếp (SORT).

Trong một vài trường hợp, các thao tác này được tổ hợp thành một thao tác phức tạp hơn. Ví dụ như thao tác SEARCH_INSERT (tìm kiếm và chèn). Thao tác này thường được dùng trong các trường hợp các mẩu tin với khoá bằng nhau không được phép lưu trữ trong cấu trúc dữ liệu. Trong nhiều phương pháp, mỗi lần xác định một khoá nào đó không có trong cấu trúc dữ liệu thì trạng thái của thủ tục tìm kiếm sẽ chứa chính xác thông tin cần thiết để chèn thêm một mẩu tin mới với khoá đã cho. Năm thao tác liệt kê trên đều có những ứng dụng rất quan trọng và một số lớn những tổ chức dữ liệu cơ sở đã được đề nghị để dùng phối hợp các thao tác trên một cách hiệu quả.

Hiện nay có rất nhiều thuật giải tìm kiếm. Tuy nhiên, trong phạm bài này chúng ta sẽ chỉ nghiên cứu phép tìm kiếm tuần tự và tìm kiếm nhị phân trong các danh sách. Sau đó, chúng ta đưa ra một sơ đồ tìm kiếm đặc biệt và mô tả cấu trúc dữ liệu cần thiết đó là cây nhị phân tìm kiếm.

6.2. Tìm kiếm tuần tự

6.2.1. Tư tưởng của thuật toán

Thuật toán tìm kiếm tuần tự (Sequential Search) là một kỹ thuật tìm kiếm đơn giản. Tư tưởng của thuật toán là: Bắt đầu từ mẩu tin đầu tiên, lần lượt so sánh khóa tìm kiếm với khóa tương ứng của các mẩu tin trong danh sách, cho tới khi tìm thấy mẩu tin có khóa bằng khóa tìm kiếm hoặc đã duyệt hết danh sách mà chưa thấy. Thuật toán tìm kiếm tuần tự dễ thực hiện nhất đối với thông tin lưu trữ dạng mảng.

6.2.2. Nội dung và cách cài đặt của thuật toán

Giả sử các mẩu tin cần tìm kiếm được lưu trữ trong một danh sách R với n mẩu tin và X là khóa tìm kiếm. Thuật toán được mô tả như sau.

```
Bước 1: Gán i = 0;//duyệt từ đầu mảng
Bước 2: while((R[i]!= X)&&(i < n))
        {
           i = i + 1;
        }
Bước 3: if(i < n) tìm thấy mẩu tin có khóa bằng X tại nút i – 1;
Bước 4: else không tìm thấy mẩu tin có khóa bằng X;
Bước 5: kết thúc.
```

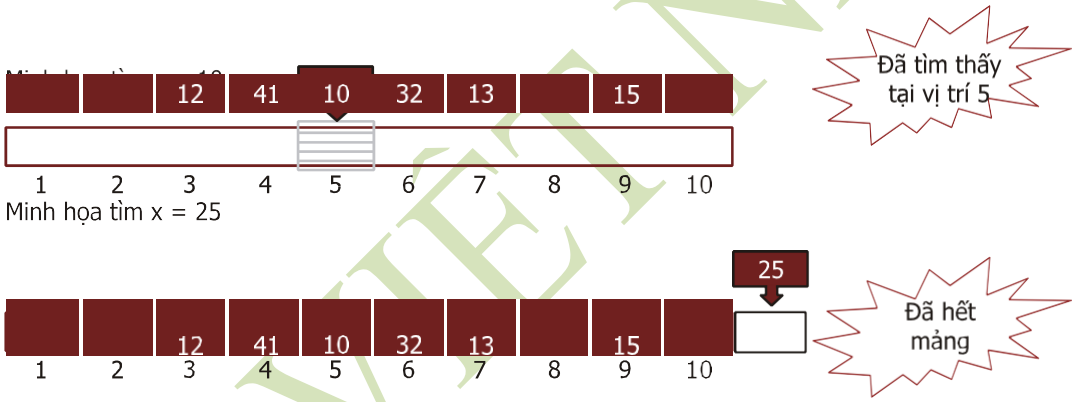
Cài đặt thuật toán:

AUM VIỆT NAM

Hàm Sequential_Search thực hiện việc tìm kiếm phần tử có giá trị X trên mảng R có n phần tử. Nếu tìm thấy, thì hàm trả về một số nguyên có giá trị tương ứng với vị trí của phần tử tìm thấy. Ngược lại, nếu không tìm thấy, hàm trả về giá trị –1.

```
typedef <kiểu_dữ_liệu> KeyType;
int Sequential_Search(dataArray R,KeyType X,int n);
    {
        int i;
        i = 0;
        while((R[i]!= X)&&(i < n))
        {
          i++;
        }
          if(i < n) return (1);
          else return(-1);
    }
```

Ví dụ 6.2. Thuật toán tìm kiếm tuần tự được minh họa trong hình dưới đây



6.2.3. Đánh giá thuật toán

- Trong trường hợp tìm kiếm tốt nhất, ngay phần tử đầu tiên của mảng có khóa bằng X khi đó
  - Số phép gán: $G_{min} = 1$
  - Số phép so sánh: $S_{min} = 2 + 1 = 3$
- Trong trường hợp tồi nhất, khi đó ta phải duyệt hết mảng thì:

  Số phép gán: $G_{max} = n$

  Số phép so sánh: $S_{max} = 2n + 1$
- Như vậy số phép toán trung bình được thực hiện trong tìm kiếm tuấn tự đó là:
  - Số phép gán: $G_{avg} = n / 2$
  - Số phép so sánh: $S_{avg} = (3 + 2n + 1) / 2 = n + 2$

Trong thuật toán trên, ở mỗi bước lặp, chúng ta cần phải thực hiện 2 phép so sánh để kiểm tra sự tìm thấy và kiểm soát sự hết mảng trong quá trình duyệt mảng. Chúng ta có thể giảm bớt đi một phép so sánh bằng cách ta thêm vào cuối mảng một phần tử "cầm canh" có giá trị bằng X để nhận diện ra sự duyệt hết mảng. Khi đó, thuật toán được cải tiến như sau:

AUM VIET NAM

```
typedef <kiểu_dữ_liệu> KeyType;
int Sequential_Search(dataArray R,KeyType X,int n);
    {
        int i;
        i = 0;
        R[n] = X;
        while(R[i]!= X)
        {
          i++;
          }
          if(i < n) return (1);
          else return(-1);
    }
```

Với việc sử dụng phần tử cầm canh ở cuối mảng tìm kiếm, thì số phép toán thực hiện trung bình trong thuật toán này là:

- Số phép gán: $G_{avg} = n / 2$
- Số phép so sánh: $S_{avg} = (n + 1) / 2$

Nếu mảng đang tìm được sắp xếp thứ tự, nghĩa là các phần tử trong mảng được sắp xếp theo thứ tự tăng dần (hay giảm dần), ta có thể xác định ngay giá trị cần tìm có nằm trong mảng hay không mà không cần phải duyệt hết tất cả các phần tử của mảng do ngay khi đạt đến một phần tử lớn hơn (hoặc nhỏ hơn) giá trị cần tìm thì việc tìm kiếm có thể kết thúc.

Ví dụ 6.3.

Khi tìm giá trị X = 35 trong mảng được sắp xếp theo thứ tự tăng dần như sau:

10, 20, 30, 40, 50, 60, 70, 80, 90, 100

Ta nhận thấy không cần thiết phải tìm vượt quá phần tử có giá trị 40 vì phần tử này và các phần tử sau nó đều có giá giá trị lớn hơn 35, và việc tìm kiếm có thể kết thúc tại đây.

Từ đây ta có thuật toán tìm kiếm tuần tự cho các danh sách (ở trường hợp này là danh sách được lưu trữ dưới dạng mảng) có thứ tự như sau:

```
typedef <kiểu_dữ_liệu> KeyType;
int LinearSearch(KeyType X, dataArray R,int n)
{
        int i;
        for(i = 0; i < n; i++)
        {
            if(R[i] == X) return(i);
            else if(X < R[i]) return(-1);
        }
}
```

Tuy nhiên, trong trường hợp tồi nhất đó là phần tử tìm thấy ở cuối danh sách có thứ tự hoặc các phần tử trong danh sách có thứ tự đều nhỏ hơn giá trị cần tìm thì số lần thực hiện phép so sánh cùng là $O(n) \approx n$.

AUM VIỆT NAM

Như vậy, tìm kiếm tuần tự là đơn giản và thuận tiện khi danh sách tìm kiếm không lớn. Tuy nhiên, trong trường hợp số phần tử của danh sách tìm kiếm là rất lớn, chẳng hạn chúng ta tìm kiếm tên một khách hàng trong danh bạ điện thoại của một thành phố lớn, thì theo tìm kiếm tuần tự sẽ mất rất nhiều thời gian. Trong thực tế rất nhiều danh sách tìm kiếm đã được sắp xếp theo một thứ tự nào đó, do vậy chúng ta sẽ tìm hiểu một số thuật toán tìm kiếm khác nhằm rút bớt thời gian tìm kiếm trên danh sách có thứ tự đó là các thuật toán tìm kiếm nhị phân.

6.3. Tìm kiếm nhị phân

6.3.1. Tư tưởng của thuật toán

Tư tưởng của thuật toán tìm kiếm nhị phân là tìm kiếm dựa trên ứng dụng sơ đồ "chia – để – trị". Điều này có nghĩa là chúng ta sẽ chia những mẩu tin trong danh sách tìm kiếm thành hai phần, xác định xem phần nào chứa khoá cần tìm. Kế đến tiếp tục công việc cho phần chứa khoá ta vừa tìm được. Việc chia đôi và tìm kiếm trên một nửa các mẩu tin chỉ có thể thực hiện được trong danh sách có thứ tự. Trong tìm kiếm nhị phân, chúng ta giả sử rằng các mẩu tin trong danh sách có khóa đã được sắp xếp theo thứ tự tăng dần (với trường hợp giảm dần ta cũng làm tương tự), tức là các phần tử đứng sau luôn có giá trị lớn hơn hoặc bằng phần tử đứng trước nó. Để tìm khóa X có trong một dãy hay không, trước tiên ta so sánh X với khóa của mẩu tin ở giữa danh sách. Nếu X nhỏ hơn thì X chỉ có thể nằm trong nửa đầu tiên của danh sách. Ngược lại, nếu X lớn hơn thì X nằm trong nửa còn lại. Lặp lại quá trình này cho đến khi tìm thấy X hoặc phạm vi tìm kiếm không còn nữa. Dưới đây ta mô tả chi tiết thuật toán tìm kiếm nhị phân.

6.3.2. Nội dung và cách cài đặt thuật toán

(Thuật toán này tìm X trong danh sách dưới dạng mảng gồm n phần tử được sắp xếp theo thứ tự tăng dần, lần lượt là: R_1, R_2,…, R_n. Nếu tìm thấy sẽ trả về vị trí của phần tử có khóa bằng X trong mảng; nếu không thì trả về giá trị mặc định bằng –1).

```
Bước 1: đặt First = 0 và Last = n - 1;
Bước 2: Found = -1;//Found là biến lưu vị trí tìm thấy X trong mảng
Bước 3: while((First <= Last)&&(Found == -1))
        {
                Mid =(First + Last)/2;
                if(X < R[Mid]) Last = Mid - 1;
                else if(X > R[Mid]) First = R[Mid] + 1;
                else Found = Mid;
        }
```

Cài đặt thuật toán:

Hàm BinarySearch dưới đây mô tả thuật toán tìm kiếm nhị phân nêu trên, trong đó hàm tìm kiếm phần tử X trong mảng R gồm n phần tử. Nếu tìm thấy hàm trả về giá trị là vị trí của X trong mảng; nếu không tìm thấy, hàm trả về giá trị –1.

AUM VIỆT NAM

```
int BinarySearch(ArrayType R,KeyType X,int n);
{
        int Mid,First,Last;
        int Found;
        First = 0;
        Last = n - 1;
        Found = -1;
        while((First <= Last)&&(Found = -1))
        {
                Mid = (First + Last)/2;
                if(X <  R[Mid])
                    Last = Mid - 1;
                else if (X > R[Mid]) First = Mid + 1;
                else Found = Mid;
        }
        return  Found;
};
```

Ví dụ 6.4.

Giả sử ta có một mảng R gồm 10 phần tử như sau:

10 20 30 40 50 60 70 80 90 100

- Xét trường hợp tìm phần tử có giá trị X = 40 (trường hợp tìm thấy) ta có:

Lần lặp	First	Last	First <= Last	Mid	R[Mid]	X = R[Mid]	X < R[Mid]	X > R[Mid]
Ban đầu	1	10	True	5	50	False	True	False
1	1	4	True	2	20	False	False	True
2	3	4	True	3	30	False	False	True
3	4	4	True	4	40	True		

Như vậy, sau 3 lần lặp, thuật toán (lặp) kết thúc và ta tìm được phần tử thứ 4 có giá trị bằng X.

- Xét trường hợp tìm phần tử có giá trị X = 45

Lần lặp	First	Last	First <= Last	Mid	R[Mid]	X = R[Mid]	X < R[Mid]	X > R[Mid]
Ban đầu	1	10	True	5	50	False	True	False
1	1	4	True	2	20	False	False	True
2	3	4	True	3	30	False	False	True
3	4	4	True	4	40	False	True	False
4	5	4	False					

Như vậy, sau 4 lần lặp, thuật toán (lặp) kết thúc.

6.3.3. Đánh giá thuật toán

Trong thuật toán này, mỗi lần đi qua vòng lặp, độ lớn của danh sách con được giảm đi để tìm trong một nửa danh sách con. Lần cuối cùng đi qua vòng lặp khi danh sách con có độ lớn là 1. Giả sử sau k lần lặp ta có danh sách con cuối cùng bằng 1. Mặt khác, do độ lớn của danh sách con sau k lần lặp là $n/2^k$, ta có:

$$\frac{n}{2^k} < 2 \text{ nghĩa là } n < 2^{k+1} \qquad \text{hay } \log_2 n < k + 1.$$

Như vậy, trong trường hợp tồi nhất độ phức tạp của thuật toán này là $O(\log_2 n)$.

Ta thấy rằng thuật toán tìm kiếm nhị phân được cài đặt ở trên là lặp, ta hoàn toàn có thể mô tả nó bằng cách đệ quy. Ý tưởng của tìm kiếm nhị phân đệ quy là tìm phần tử ở giữa danh sách (con) và nếu nó không phải là phần tử cần tìm, ta tiếp tục tìm ở một trong hai nửa của danh sách (con) theo cách hoàn toàn giống như trước.

Tìm kiếm nhị phân đề quy được cài đặt như sau:

Mô tả thuật toán:

```
Bước 1: đặt First = 0;Last = n - 1; Found = -1;
Bước 2: if (First <= Last)
        {
            Mid = (First + Last)/2;
            if (X < R[Mid])
                Áp dụng thuật toán này với Last = Mid - 1;
            else if(X > R[Mid])
                Áp dụng thuật toán này với First = Mid + 1;
            else Found = Mid;
        }
```

Cài đặt thuật toán qua xây dựng hàm tìm kiếm BinarySearch

```
int BinarySearch(KeyType, ArrayType R,int First, int Last)
{
        int Mid;
        if(First > Last) return (-1);
        else
        {
            Mid = (First + Last)/2;
            if(X < R[Mid]) BinarySearch(X, R, First,Mid - 1);
            else if(X > R[Mid]) BinarySearch(X, R, Mid + 1,Last);
                 else return(Mid);
          }
}
```

Và độ phức tạp của thuật toán tìm kiếm nhị phân đệ quy cũng là $O(\log_2 n)$.

Chú ý:

- Thuật toán tìm kiếm nhị phân chỉ có thể vận dụng trong trường hợp dãy (mảng) đã có thứ tự. Trong trường hợp tổng quát, chúng ta chỉ có thể áp dụng thuật toán tìm kiếm tuần tự.

AUM VIỆT NAM

- Các thuật toán đệ quy có thể ngắn gọn, song tốn kém bộ nhớ để ghi nhận mã lệnh chương trình (mỗi lần gọi đệ quy) khi chạy chương trình, do vậy có thể làm chương trình chạy chậm lại. Vì vậy, khi viết chương trình nếu có thể, chúng ta nên sử dụng thuật toán không đệ quy.

Phép tìm kiếm nhị phân hơn hẳn tìm kiếm tuần tự trên danh sách có thứ tự. Tuy nhiên, nó cũng đòi hỏi phải được cài đặt tuần tự trên cơ sở mảng để có thể truy cập trực tiếp tới các phần tử của danh sách. Nó không thích hợp khi cài đặt trên danh sách liên kết vì khi đó việc định vị phần tử ở giữa danh sách đòi hỏi phải đi qua danh sách con chứa các phần tử đứng trước phần tử này. Khi đó cho dù là áp dụng thuật toán tìm kiếm nhị phân cũng không mang lại nhiêu hiệu quả như tìm kiếm tuần tự vì vẫn phải duyệt qua từng phần tử. Tuy nhiên, có thể lưu trữ các phần tử của danh sách có thứ tự trong một cấu trúc liên kết để có thể tìm theo thuật toán tìm kiếm nhị phân. Cấu trúc này được gọi là cây nhị phân tìm kiếm và đó là cấu trúc dữ liệu sẽ được nghiên cứu ở phần tiếp theo của bài.

6.4. Cây nhị phân tìm kiếm

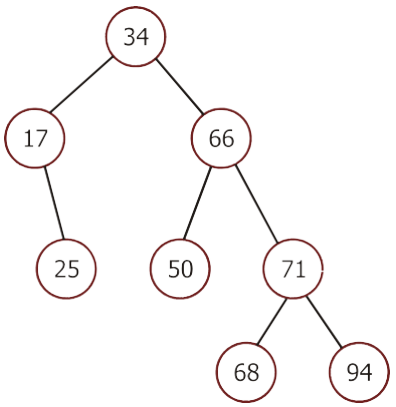
Chúng ta nhận thấy rằng danh sách liên kết là một cấu trúc dữ liệu rất có ích cho việc xử lý các danh sách động có độ dài thay đổi và các danh sách trên đó có các thao tác chèn và xóa. Tuy nhiên, việc áp dụng thuật toán tìm kiếm nhị phân trên danh sách lại không mang nhiều hiệu quả so với tìm kiếm tuần tự, mặc dù thực tế tìm kiếm nhị phân hiệu quả hơn nhiều. Trong phần này chúng ta mô tả một cấu trúc dữ liệu đặc biệt gọi là cây nhị phân tìm kiếm (Binary Search Tree). Cấu trúc dữ liệu này cho phép tìm kiếm nhị phân trên một cấu trúc liên kết.

6.4.1. Khái niệm về cây nhị phân tìm kiếm

Cây nhị phân tìm kiếm là cây nhị phân mà mỗi nút đều được gán một khóa sao cho với mỗi nút k:

- Mọi khóa trên cây con trái đều nhỏ hơn khóa trên nút k.
- Mọi khóa trên cây con phải đều lớn hơn khóa trên nút k.

Ví dụ 6.5. Hình dưới đây mô tả một cây nhị phân tìm kiếm.



Như vậy, ta hoàn toàn có thể lấy cấu trúc dữ liệu biểu diễn cây nhị phân để biểu diễn cho cây nhị phân tìm kiếm (xem bài 5). Ở phạm vi bài này, ta dùng cấu trúc liên kết để biểu diễn cho cây nhị phân tìm kiếm. Cấu trúc dữ liệu biểu diễn cây nhị phân tìm kiếm được cài đặt như sau:

AUM VIỆT NAM

```
typedef <kiểu_dữ_liệu> KeyType //kiểu của khóa trong các nút
typedef struct Node
          {
                KeyType info;
                Node *left;
                Node *right;
          }Node;
//Định nghĩa cây nhị phân
typedef Node * Tree;
Tree BST;
```

Cây nhị phân tìm kiếm là một cấu trúc dữ liệu cơ bản được sử dụng để xây dựng các cấu trúc dữ liệu trừu tượng hơn như các tập hợp, đa tập hợp, các dãy kết hợp.

6.4.2. Các thao tác trên cây nhị phân tìm kiếm

6.4.2.1. Thao tác chèn thêm một nút

Đây là thao tác chèn thêm một nút vào cây nhị phân sao cho sau khi chèn, cây vẫn là cây nhị phân tìm kiếm. Trong thao tác này, trước hết, chúng ta phải tìm kiếm vị trí cần chèn, sau đó mới tiến hành thêm nút mới vào cây. Quá trình tìm kiếm vị trí chèn giống như thao tác tìm kiếm ở trên. Thuật toán thực hiện thao tác chèn trên cây nhị phân tìm kiếm như sau:

(Thuật toán này thực hiện chèn một nút có thành phần dữ liệu là NewKey, nếu Newkey bị trùng với dữ liệu của một nút nào đó trên cây thì việc chèn sẽ không được thực hiện để tránh trùng lặp)

```
Bước 1: Tạo một nút mới NewNode có thành phần dữ liệu NewKey
Bước 2: Gán con trỏ Root = BST, PrtTree = NULL
Bước 3: Nếu( (Root == NULL) || (Root -> infor == Newkey))
                  Chuyển đến bước 8
Bước 4: Ngược lại: Gán PrtTree = Root;
Bước 5: Nếu Root -> infor > NewKey
                    Root = Root -> left;
Bước 6: Nếu Root -> infor < NewKey
                  Root = Root -> right;
Bước 7: lặp lại bước 3
Bước 8: Nếu(Root != NULL)
                      printf('Trùng khóa của một nút trên cây');
         -  Ngược lại nếu (PrtTree == NULL)
                      BST = NewNode;  // cây BST trống
         -  Ngược lại:  + Nếu (PrtTree -> infor > NewKey
                               PrtTree^.left = NewNode;
                        + Nếu (PrtTree -> infor < NewKey)
                               PrtTree -> right = NewNode;
Bước 9: Kết thúc
```

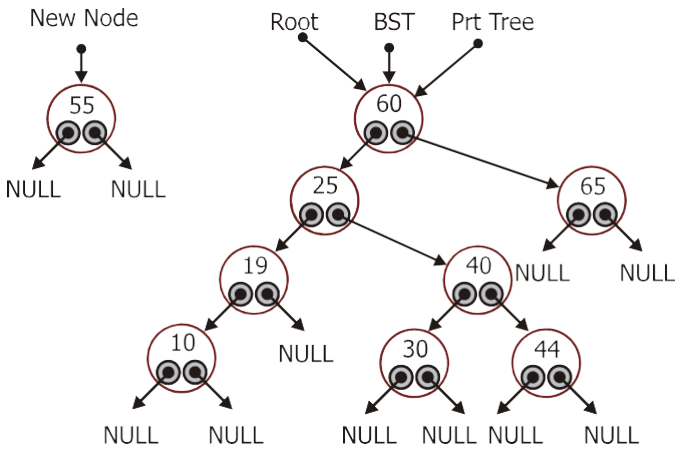
AUM VIỆT NAM

Cài đặt thuật toán chèn một nút vào cây nhị phân tìm kiếm

```
void BSTinseart (Tree BST, KeyType NewKey)
{
        Tree Root, PrtTree;
        Root = BST;
        PrtTree = NULL;
        while((Root != NULL)&&(Root -> infor != NewKey))
        {
                PrtTree = Root;
                if (Root -> infor > NewKey) Root = Root -> left;
                else if (Root -> infor < NewKey) Root = Root -> right;
        }
          if(Root != NULL)
                    printf("da co phan tu NewKey tren cay");
        else
        {
                Root = (Tree)malloc(sizeof(Tree));
                Root -> infor = NewKey;
                Root -> left = NULL;
                Root -> right = NULL;
                if(PrtTree == NULL) BST = Root;
                else if(PrtTree -> infor > NewKey)
                                PrtTree -> left = Root;
                     else    PrtTree- > right = Root;
        }
}
```

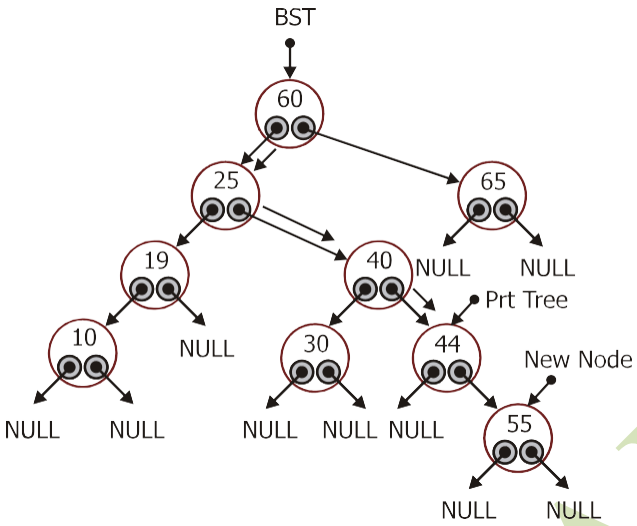
Thuật toán chèn một nút vào cây nhị phân tìm kiếm còn được gọi là thuật toán tìm kiếm và thêm vào cây.

Ví dụ 6.6. Chèn một nút mới NewNode chứa dữ liệu 55 vào cây nhị phân tìm kiếm sau:



Theo thuật toán trên việc tìm kiếm vị trí để dừng sẽ kết thúc tại nút chứa 44 mà con trỏ PrtTree trỏ đến và thực hiện việc chèn ta có kết quả như sau:

AUM VIỆT NAM



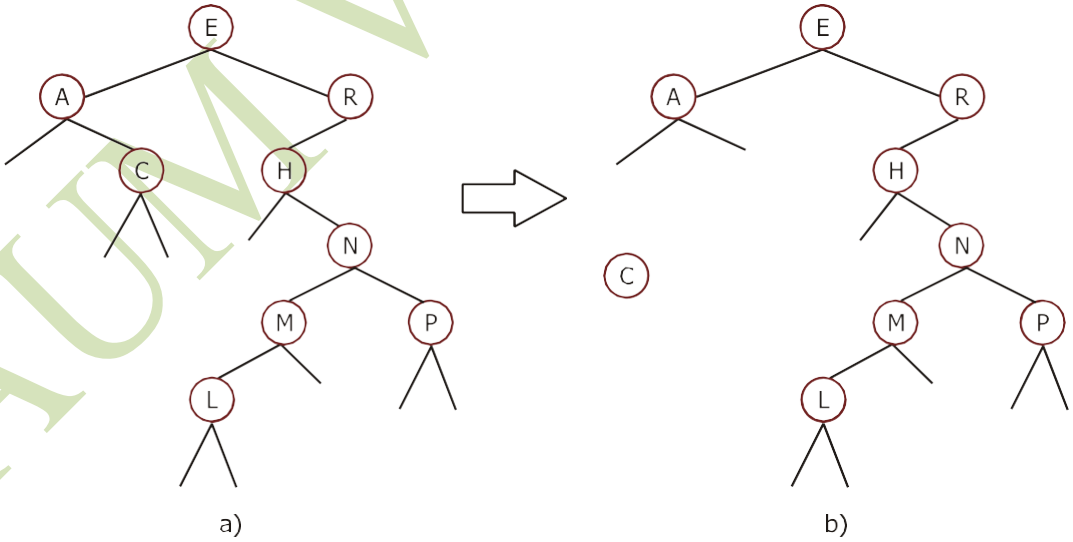
6.4.2.2. Thao tác xóa một nút trên cây nhị phân tìm kiếm

Giống như thao tác chèn một nút mới vào cây, việc xóa một nút trên cây nhị phân tìm kiếm vẫn phải bảo đảm cây sau khi xóa là cây nhị phân tìm kiếm. Đây là một thao tác không đơn giản vì nếu thiếu cẩn thận, chúng ta có thể biến cây thành một rừng. Việc xóa một nút trên cây nhị phân tìm kiếm có thể xảy ra ở một trong ba trường hợp sau:

- Nút xóa là nút lá

  Trường hợp này là đơn giản, ta chỉ cần đặt con trỏ tương ứng trong nút bố của nút cần xóa trỏ đến nul; đó là con trỏ phải hay trái tùy thuộc vào nút cần xóa là con bên phải hay bên trái.

Ví dụ 6.7. Giả xử ta xóa nút C của cây nhị phân tìm kiếm có cấu trúc sau (hình a)



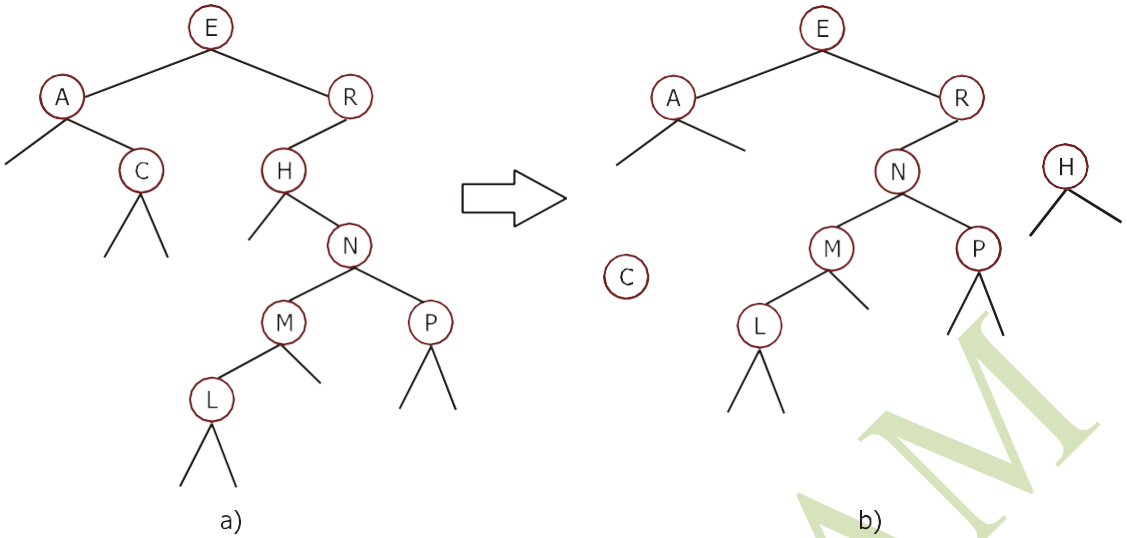
Khi đó, ta chỉ việc đặt con trỏ phải của nút A là cha của C chỉ đến nul rồi giải phóng nút C.

- Nút xóa là nút chỉ có một nút gốc cây con.

  Trường hợp này cũng đơn giản vì khi đó ta chỉ cần đặt con trỏ thích hợp ở nút bố của nút cần xóa chỉ đến con của nút cần xóa.

AUM VIỆT NAM

Ví dụ 6.8. Giả xử ta xóa nút H của cây nhị phân tìm kiếm có cấu trúc sau (hình a)

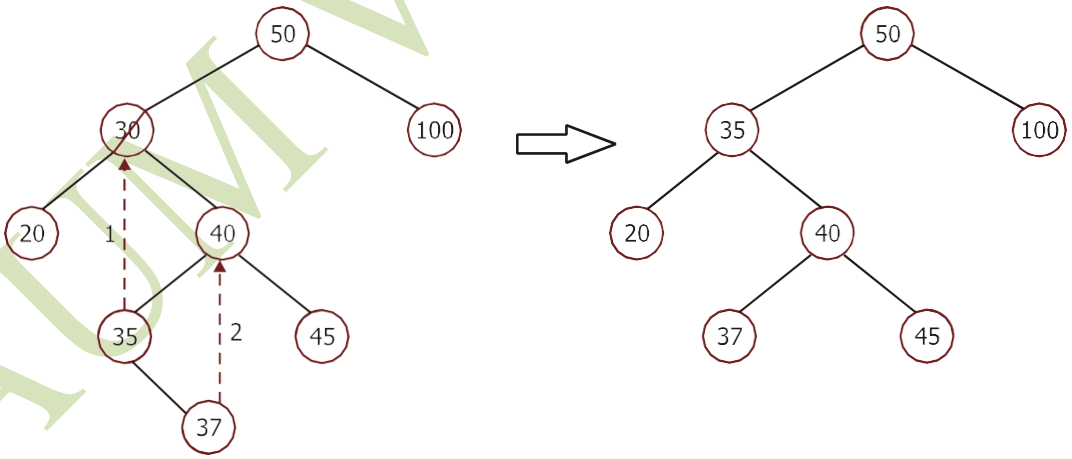


Khi đó, ta chỉ việc đặt con trỏ trái của nút R là cha của nút H chỉ đến nút N rồi giải phóng nút H ta có cây nhị phân tìm kiếm mới (hình b).

- Nút xóa có đủ hai nút gốc cây con.

  Trong trường hợp này, xóa nút đó và thay thế nó bằng nút có khóa lớn nhất trong các khóa nhỏ hơn khóa của nó (được gọi là "nút tiền nhiệm" – nút cực phải của cây con trái) hoặc nút có khóa nhỏ nhất trong các khóa lớn hơn nó (được gọi là "nút kế vị" – nút cực trái của cây con phải). Khi tìm được nút tiền nhiệm hoặc nút kế vị thì đổi chỗ nó với nút cần xóa và sau đó xóa nó. Vì các nút kiểu này có ít hơn hai con nên việc xóa nó được quy về hai trường hợp trước.

Ví dụ 6.9. Xóa nút có giá trị 30 ở cây nhị phân tìm kiếm dưới đây bằng thuật toán "tìm nút kế vị".



Trước hết, ta định vị nút kế vị bằng cách bắt đầu từ nút con bên phải của nút chứa 30, rồi đi xuống bên trái nhất có thể. Ở ví dụ này, nút kế vị là nút chứa 35. Bây giờ ta thay thế nội dung của nút chứa 30 bằng nội dung của nút chứa 35. Sau đó, tiến hành xóa nút kế vị này như trong trường hợp xóa nút có chỉ có một cây con.

Cài đặt thao tác xóa một nút cho cây nhị phân tìm kiếm: Ta xây dựng thủ tục BSTDelete() thực hiện việc tìm nút có giá trị bằng X và xóa nó từ cây nhị phân tìm kiếm với nút gốc được chỉ bởi con trỏ BST. Thủ tục này thực hiện thao tác xóa cho tất cả các trường hợp, trường hợp thứ ba được thu về hai trường hợp đầu tiên bằng cách sử dụng nút kế vị như minh họa ở ví dụ 6.8.

AUM VIỆT NAM

```
void BSTDelete(Tree BST, KeyType X)
{
        Tree Root,PrtTree,Parent;
        Root = BST;
        Parent = NULL;
        while(Root != NULL)
        {
        if(Root -> infor == X) break;
        Parent = Root;
        if(Root -> infor < X) Root = Root -> left;
        else Root = Root -> right;
        }
          if(Root == NULL)
                    printf("khong có phan tu nay tren cay");
        else
        {
                if((Root -> left != X)&&(Root -> right != X)
                {
                PrtTree = Root -> right;
                Parent = Root;
                while(PrtTree -> left != NULL)
                      {
                       Parent = PrtTree;
                       PrtTree = PrtTree -> left;
                      }
                         Root -> infor = PrtTree -> infor;
                }
                 PrtTree = Root -> left;
                 if (PrtTree == NULL)
                     PrtTree = Root -> right;
                 if (Parent == NULL)
                     Root = PrtTree;
                 else if(Parent -> left == Root)
                      Parent -> left = PrtTree;
                 else
                 Parent -> right = PrtTree;
         }
}
```

Trong cây nhị phân tìm kiếm thì các thao tác duyệt cây là tương tự như các thao tác duyệt trên cây nhị phân (tìm hiểu phần 5.2.3 của bài 5).

Trong trường hợp trung bình, thao tác xóa trên cây nhị phân tìm kiếm có độ phức tạp là O(lgn). Trong trường hợp xấu nhất, cây nhị phân tìm kiếm bị suy biến thì thao tác này có độ phức tạp là O(n) với n là số nút trên cây nhị phân tìm kiếm.

AUM VIỆT NAM

6.4.2.3. Tìm kiếm trên cây nhị phân tìm kiếm

Việc tìm một khóa trên cây nhị phân tìm kiếm có thể thực hiện bằng thuật toán tìm kiếm nhị phân. Chúng ta bắt đầu từ gốc. Nếu khóa cần tìm bằng khóa của gốc thì khóa đó trên cây, nếu khóa cần tìm nhỏ hơn khóa ở gốc, ta phải tìm nó trên cây con trái; nếu khóa cần tìm lớn hơn khóa ở gốc, ta phải tìm nó trên cây con phải. Nếu cây con (trái hoặc phải) là rỗng thì khóa cần tìm không có trên cây.

Thuật toán thực hiện thao tác tìm kiếm trên cây nhị phân tìm kiếm được mô tả như sau: (Trong thuật toán này, chúng ta tìm kiếm trên cây nhị phân có nút nào có khóa bằng giá trị X hay không? Nếu tìm thấy, trả về con trỏ Root trỏ tới nút chứa khóa cần tìm, ngược lại trả về con trỏ Root trống)

```
Bước 1: đặt con trỏ Root = BST;
Bước 2: nếu (Root = NULL) hoặc (Root -> infor = X)
                Kết thúc thuật toán;
Bước 3: ngược lại:nếu (Root -> infor > X)
                    Root = Root -> left;//tìm X ở cây con bên trái
Bước 4: ngược lại nếu(Root -> infor < X)
                    Root = Root -> right;//tìm X ở cây con bên phải
Bước 5: lặp lại bước 2;
```

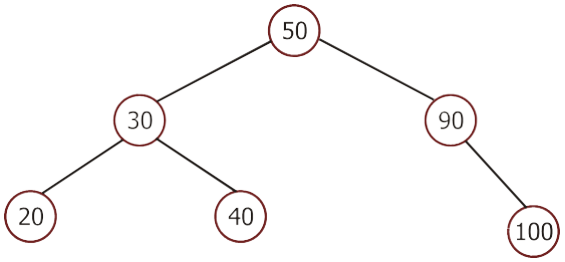
Như vậy, thuật toán kết thúc việc tìm kiếm khi cây con trống hoặc nút gốc của cây con có giá trị khóa cần tìm.

Cài đặt thuật toán:

```
Tree BSTSearch(Tree BST,KeyType X)
{
        Tree Root;
        Root = BST;
        while((Root != NULL)&&(Root - >infor != X))
        {
                if(Root -> infor > X) Root = Root -> left;
                else if(Root -> infor < X) Root = Root -> right;
        }
        return Root;
}
```

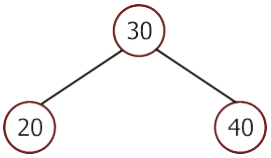
Cũng như trong trường hợp tìm kiếm nhị phân ở phần trước, việc cài đặt thuật toán thực hiện thao tác tìm kiếm trên cây nhị phân tìm kiếm cũng có thể được cài đặt theo kiểu đệ quy. Việc trình bày và cài đặt thuật toán tìm kiếm nhị phân trên cây nhị phân tìm kiếm được xem như là bài tập để học viên tự giải.

Ví dụ 6.10. Xét cây nhị phân tìm kiếm có dạng sau:



AUM VIỆT NAM

Ta tìm nút có thành phần dữ liệu là 40 trên cây nhị phân tìm kiếm này. Theo thuật toán trên ta tiến hành như sau: bắt đầu từ nút gốc, và vì 40 nhỏ hơn giá trị 50 của nút gốc nên chúng ta biết rằng nút cần tìm ở bên trái của nút gốc, nghĩa là nó phải ở bên trong cây con bên trái, có nút gốc là 30:



Ta tiếp tục tìm kiếm bằng cách so sánh 40 với gốc của cây con này. Vì 40 > 30 nên nút cần tìm sẽ nằm ở cây con bên phải đó là:



Thử so sánh 40 với giá trị ở nút gốc của cây con chỉ có một nút này, ta tìm được nút chứa giá trị cần tìm.

Chú ý

Nếu cây nhị phân tìm kiếm là cây nhị phân hoàn chỉnh thì thời gian tìm kiếm trung bình là O(log n), còn là cây nhị phân lệch phải (trái) thì thời gian tìm kiếm trung bình là O(n).

AUM VI... NAM

TÓM LƯỢC CUỐI BÀI

Tìm kiếm là công việc được sử dụng nhiều trong các ứng dụng. Trên đây, ta đã trình bày phép tìm kiếm trong một danh sách để tìm ra mẩu tin có khóa đúng bằng khóa tìm kiếm. Tuy nhiên, trong nhiều trường hợp ta có thể chuyển sang tìm mẩu tin mang khóa lớn hơn hay nhỏ hơn khóa tìm kiếm, tìm mẩu tin mang khóa nhỏ nhất mà lớn hơn khóa tìm kiếm v.v… Để cài đặt những thuật toán cho các trường hợp này cần có sự mềm dẻo nhất định. Ngoài ra, chúng ta cũng không nên đánh giá giải thuật tìm kiếm này tốt hơn giải thuật tìm kiếm khác mà căn cứ vào từng yêu cầu cụ thể để đưa ra giải thuật tìm kiếm phù hợp nhất.

AUM VIỆT NAM

BÀI TẬP

1. Trình bày tư tưởng của các thuật toán tìm kiếm: tuần tự và nhị phân? Các thuật toán này có thể được vận dụng trong các trường hợp nào? Cho ví dụ?

2. Cài đặt lại thuật toán tìm kiếm tuần tự bằng các cách:

– Sử dụng vòng lặp for

– Sử dụng vòng lặp Repeat… until

Cho nhận xét?

3. Đối với mỗi danh sách các ký tự dưới đây, hãy vẽ cây nhị phân tìm kiếm với các ký tự được chèn vào theo thứ tự sau:

a. A, C, R, E, S

b. R, A, C, E, S

c. C, A, R, E, S

d. S, C, A, R, E

e. C, O, R, N, F, L, A, K, E

4. Nhập vào một dãy gồm n số nguyên dương với n nhập vào từ bàn phím. Nhập một giá trị X. Viết chương trình xác định X có xuất hiện trong dãy không? Nếu có, đưa ra các vị trí mà X xuất hiện trong dãy.

5. Viết chương trình nhập vào từ bàn phím 1 dãy số nguyên, lưu trữ nó trong 1 danh sách có thứ tự tăng không có 2 phần tử trùng nhau, theo cách sau: với mỗi phần tử được nhập vào chương trình phải tìm kiếm xem nó có trong danh sách chưa? Nếu chưa, xen nó vào danh sách cho đúng thứ tự.

AUM VIỆT NAM